

Nuestra Cafetería

David Priego Puga

October 29, 2025

1 Introducción

En este proyecto pequeño donde seguiremos aprendiendo a trabajar con **hilos** en **java** tenemos que construir una **cafetería** con camareros y clientes. Donde nuestros camareros tendrán que atender a los diferentes que lleguen a la **cafetería**. Los clientes por otro lado cuando llegan a nuestro local tienen un tiempo de espera el cual es aleatorio (Esto está hecho de esta manera para darle un añadido de aleatoriedad y comprobar que los hilos funcionan bien debido a que cada ejecución del programa es completamente aleatoria). Lo único que no es aleatorio es el orden de llegada de los clientes y los camareros.

Por otro lado volviendo a los clientes, estos pueden llegar a la cafetería, ser atendidos dentro de su **tiempo de espera** e irse con el café, por otro lado pueden llegar a la cafetería, ser atendidos pero que el camarero tarde demasiado en servirles entonces se irán enfadados y sin café y por último es que directamente llegue a la cafetería y pase el tiempo y los camareros no lleguen a atender a este cliente de tal manera que no le dará tiempo ni a pedir un café. Esto en resumidas cuentas es el resumen de nuestro programa.

2 Arquitectura de nuestro programa

Nuestro programa se basa en un código realizado en java de tal manera que que dispondremos de **3 clases diferentes: Camarero, Cliente y Main**. De tal manera que solo con estas clases se consiga la completa gestión de la cafetería.

Además como es un programa sencillo no disponemos de backend ya que usamos una **Lista** de clientes embebida dentro de nuestro **JavaFX**.

2.1 Clase Cliente

En esta clase definimos a cada uno de nuestros clientes. Tenemos dos atributos para cada uno de nuestros **clientes**, cada cliente tiene **nombre** y **tiempo de espera**.

- Nombre: Atributo **String** que indica como se llama el cliente.
- TiempoEspera: Atributo **Int** el cual se calcula mediante un **Math.random** el cual calcula un tiempo aleatorio entre 3 y 4 segundos.

Por otro lado tenemos los diferentes **getters** para cada cliente de tal manera que tenemos:

- getNombre: Método que nos devuelve el nombre del cliente,
- getTiempoEspera: Método que nos devuelve el **TiempoEspera** en un formato **Int**.

Con esto podemos ver que realmente la clase **Cliente** es una clase sencilla que lo único que hace es guardar los datos mínimos para que el programa tenga sentido en relación al cliente.

2.2 Clase Camarero

En esta clase definiremos a cada uno de los posibles camareros que tenga la cafetería pero además de que realmente quien realiza todo el "trabajo" es la clase **Camarero** ya que como hemos visto antes, cliente solo manda los datos mínimos imprescindibles.

En caso de los camareros tenemos bastantes más atributos y métodos.

Los atributos de la clase **Camarero** son:

- **String nombre:** De la misma manera que un cliente tiene un nombre en este caso tenemos un nombre para cada uno de los camareros.
- **Cliente[] clientes:** La propia lista de **clientes** se pasa como atributo para la clase camarero de tal manera que así sabe cada camarero cuantos clientes tiene. Podrían ser listas separadas pero para este **proyecto** usaremos la misma lista para todos los camareros.
- **Int indice:** Este número se encargara de marcar el progreso en un **método** posterior llamado **siguienteCliente** de tal manera que el **indice** marca el número en la lista del cliente actual. Por ejemplo el cliente con indice 0 es el cliente en posición 0 del Array por lo tanto el primero.
- **Object lock:** Es un objeto candado que usamos en el método "**siguienteCliente**" de tal manera que se usa como **bloqueo** debido a que los dos camareros interactúan con la misma lista esto hace que cuando un camarero está trabajando con la lista y aumentando el indice, el **lock** prohíbe al resto de camareros entrar en la lista de clientes y modificar el **indice**.
- **VBox contenedorMensajes:** Contenedor de cada camarero donde vamos a imprimir cada uno de los mensajes que se generan en nuestro programa de tal manera que sabemos a que camarero pertenece cada uno de los mensajes de cada cliente.

Con esto tenemos todos los atributos que dispone cada uno de los camareros, como podemos ver son un par más que los que dispone la clase cliente pero sobretodo el peso que estos tienen a la hora de ejecutar el programa.

Pero bueno ahora hablaremos de los diferentes métodos que dispone la clase **Camarero**:

- **siguienteCliente:** Método que sirve para moverse a través de la lista de clientes de tal manera que solo un **camarero** puede estar a la vez, esto se hace para evitar que ambos camareros atiendan al mismo cliente, para lograr esto se hace una comprobación de indices, primero de todo, se bloquea el acceso de otro camarero con **synchronized (lock)**, luego se comprueba que el indice actual sea menor que la longitud de la lista de clientes. Se recoge el cliente en posición = indice, el indice crece en una unidad y se devuelve el cliente. De tal manera que el primer camarero accederá al cliente en posición cero (Es decir al primer cliente) y el segundo camarero cuando entre al método **siguienteCliente** entrará cuando **indice** sea ya uno por lo que accederá al segundo cliente.
- **mostrarMensaje:** Método que se encarga de imprimir en pantalla los diferentes mensajes que se generan a la hora de atender a los clientes, debido a que en **JavaFX** tenemos un contenedor para cada camarero y ahí imprimimos dichos mensajes.
- **prepararCafe:** Este método se encarga de la gestión de los clientes y de comprobar si han sido atendidos a tiempo por el camarero para ello, se encarga de mostrar diferentes mensajes conforme el **cliente** es atendido en tiempo, si ha recibido su café o directamente se ha ido sin ni si quiera ser atendido. Para ello trabajamos con el **tiempoEspera** y el **tiempoPreparación** el cual no deja de ser otro tiempo random que ronda entre los 4 y los 5 segundos de tal manera que el camarero puede llegar a fallar el tiempo de espera de los clientes.
- Tenemos por último una comprobación de cliente, si cliente es igual a **siguienteCliente** pero es distinto a **null** entonces llamaremos a **prepararCafe** si no se da esta condición implica que la lista de clientes se terminó por lo que mostramos un mensaje donde se indica que el **camarero terminó el turno**.

2.3 Clase CafeController

Esta clase en **JavaFX** es la que substituye a la clase **Main** en nuestro programa en **Java**. Pero de base lo que tenemos en este archivo es:

- El **VBox** que nos los contenedores para cada uno de nuestro contenedores.

Aparte tenemos 3 métodos:

- **iniciarSimulación**: Método donde tenemos definido nuestro array de **clientes**, además creamos las tarjetas para cada uno de los camareros que queramos crear, en este caso tenemos 2 camareros creados.
- **crearTarjeta**: Método que crea el contenedor para la tarjeta de cada uno de los camareros.
 - Contenedor Tarjeta **VBox**
 - Label con el nombre del camarero.
 - Contenedor **VBox** que es el contenedor de los diferentes mensajes.
 - Luego a **Tarjeta** añadimos el label del nombre del camarero y el contenedor de mensajes
 - Por último creamos el camarero con su constructor y iniciamos el **hilo**.

3 Complicaciones

Para crear el programa en **JavaFx** desde los archivos básicos de java en **Intelij IDEA** tuve que usar **Platform** debido a que **JavaFX** solo dispone de un **Hilo** para mantener la coherencia y evitar que dos hilos distintos modifiquen la interfaz de tal manera que está se rompa, por lo tanto directamente tenemos que usarlo para marcar a la interfaz un "**orden de turnos**" para que cada hilo espere a que acabe de actuar el hilo actual en funcionamiento.